MLDL EXPERIMENT 7

Aim:
Build an Artificial Neural Network (ANN) using Keras/TensorFlow on the EMNIST Letters dataset.

Dataset Description:
The EMNIST Letters Dataset used in this experiment is a built-in dataset provided through TensorFlow Datasets (tfds). It is a benchmark dataset for handwritten letter recognition that contains grayscale images of uppercase and lowercase English letters from a to z. The dataset is preprocessed, normalized, and packaged inside TensorFlow Datasets, allowing direct loading without manual downloading. It is a supervised learning dataset where each 28x28 pixel image is paired with a correct letter label (a-z), making it ideal for training Artificial Neural Networks and evaluating multi-class classification performance. Due to its standardized format, balanced classes, and moderate size, the EMNIST Letters dataset is widely used for academic experiments, neural network demonstrations, and hyperparameter tuning.

- **File Type:** Built-in TensorFlow Dataset (NumPy arrays loaded via tfds)
- **Dataset Size:** Approximately 145,600 images (training + testing splits)
- **Image Dimension:** 28 x 28 grayscale pixels
- **Number of Classes:** 26 letters (a-z)

Column Name	Data Type	Description
Image Pixels	Numerical (Integer 0-255)	28x28 grayscale pixel intensity values representing the handwritten letter.
Label	Numerical (Integer 0-25)	True letter class corresponding to each image (a-z).

Dataset Source: https://www.tensorflow.org/datasets/catalog/emnist

Theory:

Artificial Neural Networks (ANN) are supervised learning models inspired by the biological structure of the human brain. An ANN consists of interconnected layers of neurons that transform input data through weighted connections and nonlinear activation functions. The primary objective of an ANN is to approximate a mapping function between input features and output labels by minimizing prediction error. ANNs are particularly effective for pattern recognition, classification, and regression tasks due to their ability to model complex nonlinear relationships. Given an input dataset:

$$X = \{x1, x2, x3, ..., xn\}, \quad xi \text{ in } R^d$$

where:

- n is the number of data points
- d is the number of features

Each neuron computes a weighted sum of inputs followed by an activation function:

$$z = SUM(wi * xi) + b$$
$$a = sigma(z)$$

where:

- wi = weight associated with input xi
- b = bias term
- z = linear combination of inputs
- sigma(z) = activation function
- a = neuron output

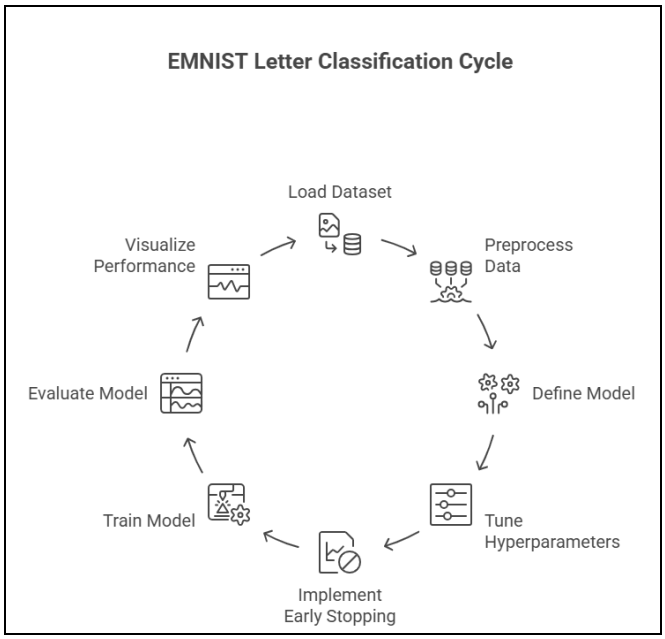
Common activation functions include:

$$ReLU(z) = max(0, z)$$
$$Sigmoid(z) = 1 / (1 + e^{-z})$$
$$Softmax(zj) = e^{zj} / SUM(e^{zk})$$

Limitations:

1. **Requirement of Large Training Data:** Artificial Neural Networks require a substantial amount of labeled training data to perform effectively. When the dataset is small, the model may fail to generalize and instead memorize the training examples, leading to overfitting.
2. **High Computational Cost:** Training an ANN involves repeated forward and backward propagation across multiple layers, which requires significant computational power and memory. Deep networks with many parameters may take hours or even days to train without specialized hardware such as GPUs or TPUs.
3. **Lack of Interpretability (Black-Box Nature):** ANN models are often criticized for being difficult to interpret. Unlike linear models or decision trees, where relationships between variables are explicit, neural networks learn complex hidden representations that are not easily explainable.

Workflow:



1. **Dataset Loading:** The EMNIST Letters dataset is loaded directly using TensorFlow Datasets (tfds). It contains handwritten grayscale images of 26 English letters (a-z), each sized 28x28 pixels.
2. **Data Preprocessing:** Pixel values are normalized from 0-255 to 0-1 by dividing by 255. Labels are shifted from 1-indexed (1-26) to 0-indexed (0-25) to align with standard Keras conventions.
3. **Model Architecture Definition:** A Sequential ANN model is defined with a Flatten input layer, a fully connected hidden layer with tunable neuron count and ReLU activation, and a Softmax output layer with 26 neurons for multi-class letter classification.
4. **Hyperparameter Tuning (Keras Tuner):** The Hyperband algorithm automatically searches for optimal hyperparameters, exploring different values for hidden neurons (32-512) and learning rate to maximize validation accuracy.
5. **Early Stopping Strategy:** An early stopping callback monitors validation loss and stops training if performance stops improving, preventing overfitting and reducing unnecessary computation.
6. **Model Training:** The optimal model (480 neurons, learning rate 0.001) is trained on 80% of training data, with 20% used for validation monitoring.
7. **Model Evaluation:** The trained ANN is evaluated on the unseen test dataset to measure final classification loss and accuracy.
8. **Performance Visualization:** Training and validation accuracy are plotted across epochs. A confusion matrix, misclassified images, sample prediction, and correct predictions are also visualized.

Performance Analysis:

The performance of the hyperparameter-tuned Artificial Neural Network is evaluated using validation accuracy, test accuracy, confusion matrix analysis, and training dynamics. Since EMNIST Letters is a supervised classification dataset with known labels, evaluation focuses on prediction accuracy, generalization capability, and class-wise performance across all 26 letter categories.

1. **Hyperparameter Optimization Results:** The Hyperband tuner explored multiple ANN configurations across 30 trials and achieved a best validation accuracy of 89.89%. The optimal architecture consists of 480 neurons in the hidden layer with a learning rate of 0.001, balancing learning capacity and generalization.
2. **Training vs Validation Accuracy Behavior:** The learning curve shows steady improvement in training accuracy, reaching approximately 95.24% by epoch 10.

Validation accuracy stabilizes around 89-90%. The gap between training and validation curves indicates mild overfitting, expected given the more complex 26-class problem.

3. **Test Set Performance:** Evaluation on the unseen test dataset yielded a Test Accuracy of 88.04% and a Test Loss of 0.4431. This consistency with validation results confirms the model generalizes well to new handwritten letter images.
4. **Confusion Matrix Analysis:** The confusion matrix reveals strong diagonal dominance for most letter classes. Errors are concentrated among visually similar pairs such as i & l, g & q, and p & f. Letters t through z showed very few test samples, indicating class imbalance in the test split.

Hyperparameter Tuning:

Artificial Neural Networks are sensitive to hyperparameter choices, which affect training stability, accuracy, and generalization. In this experiment, automated hyperparameter tuning is performed using Keras Tuner with the Hyperband algorithm, which searches multiple configurations and selects the best model based on validation accuracy.

- **Number of Hidden Neurons:** The number of neurons in the hidden layer is tuned within a range of 32 to 512 in steps of 32. The tuner selected 480 neurons as optimal, providing sufficient capacity to learn the complex structure of 26 letter classes.
- **Learning Rate of the Optimizer:** Candidate values 0.01, 0.001, and 0.0001 were tested. The value 0.001 was chosen as it provides stable convergence and efficient learning for the Adam optimizer.
- **Hyperband Search Strategy:** Hyperband efficiently discards weak models and focuses on promising configurations, enabling effective tuning across 30 trials in approximately 14 minutes.
- **Early Stopping Regularization:** Early stopping monitors validation loss and halts training when improvement stops, preventing overfitting and reducing unnecessary computation.
- **Validation-Based Selection:** Each model is evaluated using a 20% validation split. The configuration achieving the highest validation accuracy of 89.89% was selected as the final tuned model.

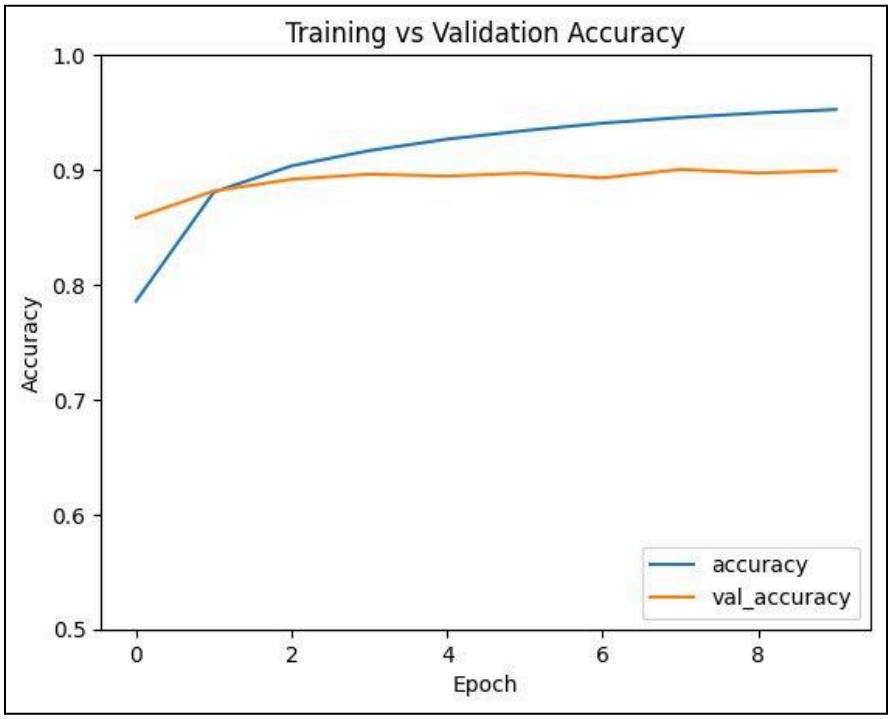
Code:

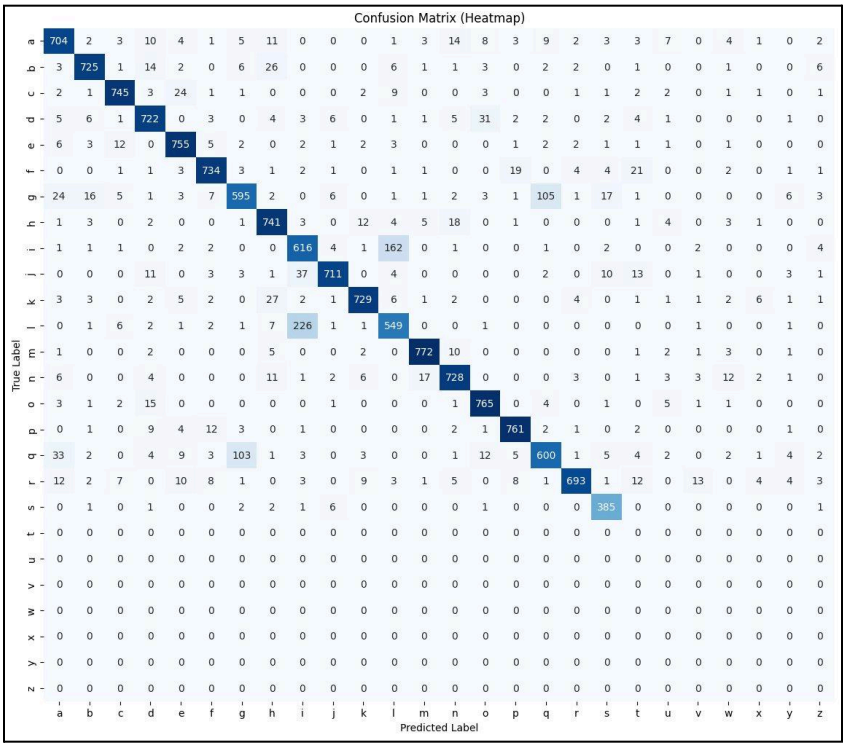
```
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix

print(f"TensorFlow Version: {tf.__version__}")

# ── 1. Load EMNIST Letters ──────────────────────────────────────────────────────────────────────────────
# emnist is not built into keras, so we use tensorflow_datasets
import tensorflow_datasets as tfds

ds_train_full, ds_test = tfds.load(
    'emnist/letters',
    split=['train', 'test'],
    as_supervised=True
)

def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = tf.squeeze(image, axis=-1)          # (28,28,1) → (28,28)
    label = label - 1                           # EMNIST letters are 1-indexed (1-26) → 0-25
    return image, label
```

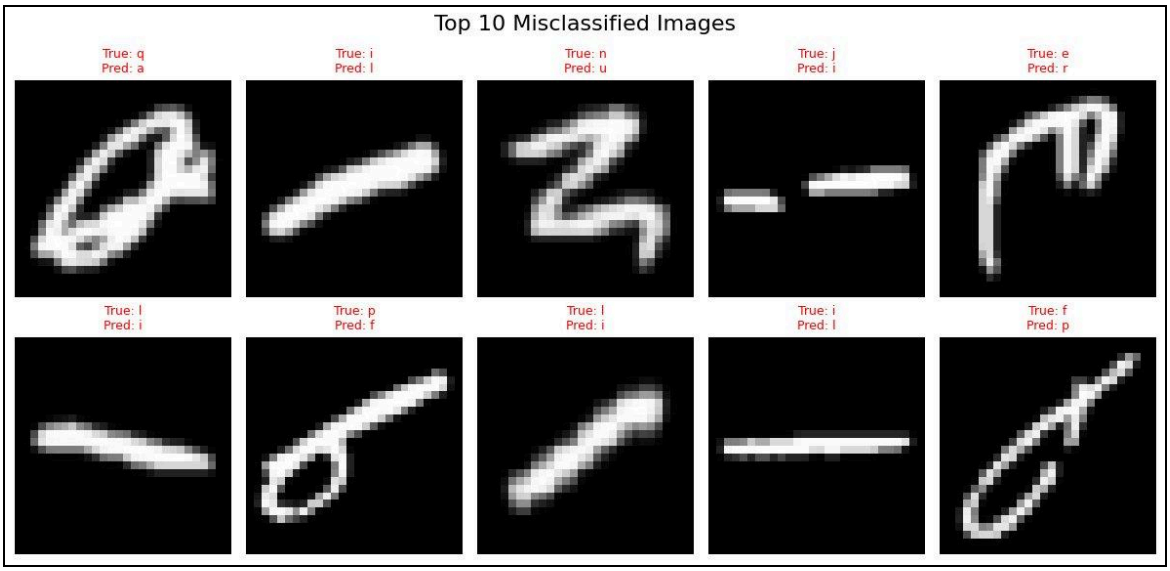
```
ds_train_full = ds_train_full.map(preprocess)
ds_test       = ds_test.map(preprocess)

# Convert to numpy for Keras
x_train = np.array([x.numpy() for x, _ in ds_train_full])
y_train = np.array([y.numpy() for _, y in ds_train_full])
x_test  = np.array([x.numpy() for x, _ in ds_test])
y_test  = np.array([y.numpy() for _, y in ds_test])

print(f"Train shape: {x_train.shape}, Test shape:
{x_test.shape}")
print(f"Classes: {len(np.unique(y_train))} letters (a–z)")

# ── 2. Model Builder for Keras Tuner
──────────────────────────────────────────────────────────────────────────────
def model_builder(hp):
    model = keras.Sequential()
    model.add(keras.layers.Flatten(input_shape=(28,
28)))
    hp_units = hp.Int('units', min_value=32,
max_value=512, step=32)
    model.add(keras.layers.Dense(units=hp_units,
activation='relu'))
    model.add(keras.layers.Dense(26,
activation='softmax'))   # 26 letter classes
    hp_lr = hp.Choice('learning_rate', values=[1e-2,
1e-3, 1e-4])
    model.compile(

optimizer=keras.optimizers.Adam(learning_rate=hp_lr
),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# ── 3. Hyperband Tuner
──────────────────────────────────────────────────────────────────────────────
tuner = kt.Hyperband(
    model_builder,
    objective='val_accuracy',
    max_epochs=10,
    factor=3,
    directory='my_dir_emnist',
    project_name='emnist_letters_tuning'
)

stop_early =
tf.keras.callbacks.EarlyStopping(monitor='val_loss',
patience=5)

tuner.search(x_train, y_train, epochs=10,
validation_split=0.2, callbacks=[stop_early])

best_hps =
tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"""
Hyperparameter search complete.
Optimal units in hidden layer : {best_hps.get('units')}
Optimal learning rate         :
{best_hps.get('learning_rate')}
""")

# ── 4. Train Best Model
──────────────────────────────────────────────────────────────────────────────
model = tuner.hypermodel.build(best_hps)
history = model.fit(x_train, y_train, epochs=10,
validation_split=0.2)

# ── 5. Evaluate
──────────────────────────────────────────────────────────────────────────────
eval_result = model.evaluate(x_test, y_test)
print(f"[Test Loss, Test Accuracy]: {eval_result}")

# ── 6. Accuracy Plot
──────────────────────────────────────────────────────────────────────────────
plt.figure()
plt.plot(history.history['accuracy'],     label='accuracy')
plt.plot(history.history['val_accuracy'],
label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.title('Training vs Validation Accuracy')
plt.savefig('accuracy_plot.png', dpi=150,
bbox_inches='tight')
plt.show()

# ── 7. Confusion Matrix
──────────────────────────────────────────────────────────────────────────────
y_pred_probs   = model.predict(x_test)
y_pred_classes = np.argmax(y_pred_probs, axis=1)

conf_matrix = confusion_matrix(y_test,
y_pred_classes)
plt.figure(figsize=(14, 12))
sns.heatmap(conf_matrix, annot=True, fmt='d',
cmap='Blues', cbar=False,
            xticklabels=list('abcdefghijklmnopqrstuvwxyz'),
            yticklabels=list('abcdefghijklmnopqrstuvwxyz'))
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix (Heatmap)')
plt.savefig('confusion_matrix.png', dpi=150,
bbox_inches='tight')
plt.show()

# ── 8. Top 10 Misclassified
──────────────────────────────────────────────────────────────────────────────
errors = (y_pred_classes != y_test)
x_err  = x_test[errors]
y_err  = y_test[errors]
p_err  = y_pred_classes[errors]
letters = list('abcdefghijklmnopqrstuvwxyz')
```

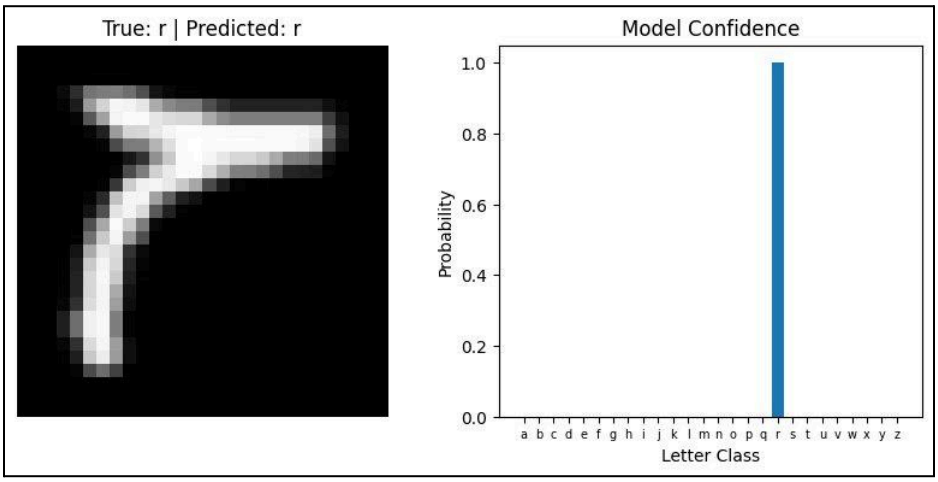
```
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_err[i], cmap='gray')
    plt.title(f"True: {letters[y_err[i]]}\nPred:
{letters[p_err[i]]}", color='red', fontsize=9)
    plt.axis('off')
plt.suptitle("Top 10 Misclassified Images",
fontsize=16)
plt.tight_layout()
plt.savefig('misclassified.png', dpi=150,
bbox_inches='tight')
plt.show()

# ── 9. Random Sample Prediction
──────────────────────────────────────────────────────────────
index        = np.random.randint(0, len(x_test))
sample_image = x_test[index]
true_label   = y_test[index]
prediction   = model.predict(sample_image.reshape(1,
28, 28))
pred_label   = np.argmax(prediction)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(sample_image, cmap='gray')
plt.title(f"True: {letters[true_label]} | Predicted:
{letters[pred_label]}")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.bar(range(26), prediction[0])
plt.xticks(range(26),
list('abcdefghijklmnopqrstuvwxyz'), fontsize=7)
plt.xlabel("Letter Class")
plt.ylabel("Probability")
plt.title("Model Confidence")
plt.savefig('sample_prediction.png', dpi=150,
bbox_inches='tight')
plt.show()

# ── 10. Hall of Fame (Top 10 Correct)
──────────────────────────────────────────────────────────────
correct_indices = np.where(y_pred_classes ==
y_test)[0]
plt.figure(figsize=(12, 6))
for i in range(10):
    idx = correct_indices[i]
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[idx], cmap='gray')
    plt.title(f"True: {letters[y_test[idx]]} | Pred:
{letters[y_pred_classes[idx]]}",
              color='green', fontsize=9)
    plt.axis('off')
plt.suptitle("Hall of Fame: Top 10 Correct Predictions",
fontsize=16)
plt.tight_layout()
plt.savefig('correct_predictions.png', dpi=150,
bbox_inches='tight')
plt.show()

# ── 11. Print summary for document
──────────────────────────────────────────────────────────────
print("\n===== COPY THIS FOR DOCUMENT
=====")
print(f"Best units          : {best_hps.get('units')}")
print(f"Best learning rate  :
{best_hps.get('learning_rate')}")
print(f"Test Loss           : {eval_result[0]:.4f}")
print(f"Test Accuracy       : {eval_result[1]*100:.2f}%")
print("==================================")
```

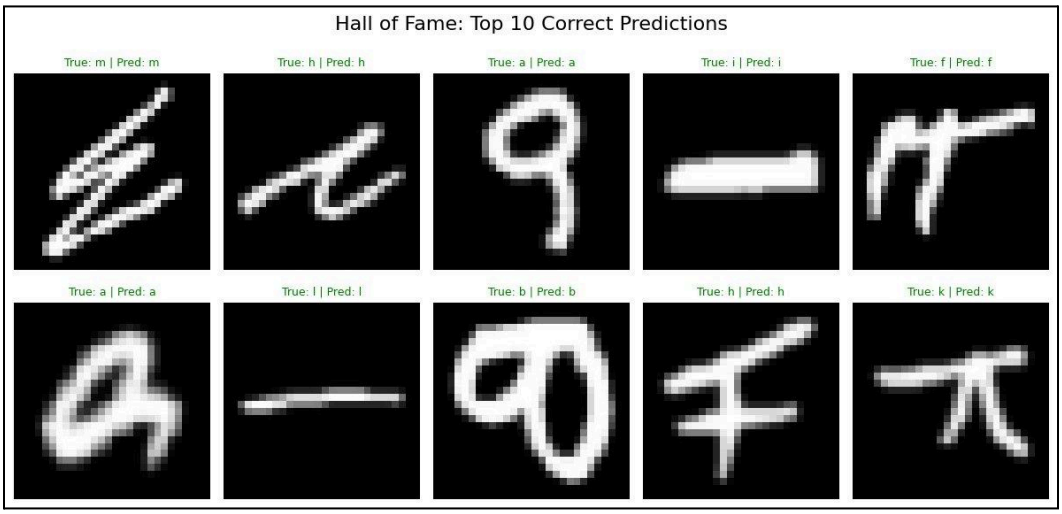
Output:


Training vs Validation Accuracy
Accuracy
Epoch
accuracy
val_accuracy
1.0
0.9
0.8
0.7
0.6
0.5
0
2
4
6
8


Confusion Matrix (Heatmap)
True Label
Predicted Label


Top 10 Misclassified Images
True: q
Pred: a
True: i
Pred: l
True: n
Pred: u
True: j
Pred: i
True: e
Pred: r
True: l
Pred: i
True: p
Pred: f
True: l
Pred: i
True: i
Pred: l
True: f
Pred: p


True: r | Predicted: r
Model Confidence
Probability
1.0
0.8
0.6
0.4
0.2
0.0
a b c d e f g h i j k l m n o p q r s t u v w x y z
Letter Class


Hall of Fame: Top 10 Correct Predictions
True: m | Pred: m
True: h | Pred: h
True: a | Pred: a
True: i | Pred: i
True: f | Pred: f
True: a | Pred: a
True: l | Pred: l
True: b | Pred: b
True: h | Pred: h
True: k | Pred: k

Conclusion:

This experiment successfully implemented a hyperparameter-tuned Artificial Neural Network using the EMNIST Letters dataset for handwritten letter classification. The tuned model achieved a best validation accuracy of 89.89% with an optimal architecture of 480 hidden neurons and a learning rate of 0.001. The final test accuracy of 88.04% demonstrates strong generalization to unseen letter images across all 26 classes.Performance analysis and confusion matrix evaluation confirmed strong classification across most letter classes, with expected misclassifications concentrated among visually similar letter pairs such as i & l, g & q, and p & f. The experiment highlights the effectiveness of automated hyperparameter tuning using the Hyperband algorithm in improving ANN performance and training efficiency. Overall, the results validate ANN as a reliable approach for handwritten letter recognition tasks, while also highlighting the additional challenge posed by 26-class letter recognition compared to digit-only datasets like MNIST.